

LangChain & LangGraph — Interview Questions

Reproduced exactly as originally written from LangChain/LangChain.md — nothing reworded.

Curated from current (2026) LangChain and LangGraph interview question banks, calibrated to what's expected from someone with roughly four years of AI engineering experience. Every answer below follows a two-layer format: a plain-language explanation with an everyday example (and a diagram wherever a picture helps), followed by a crisp, technically precise version worth saying out loud in the room. Try answering out loud before reading the model answer.

 **Interview Q1:** "What's the actual difference between a chain and an agent in LangChain?"

✅ **Strong answer:** "A chain is like a recipe — every step is written down ahead of time, and it always runs in that exact order: prompt, then model, then parser, done. An agent is different: instead of a fixed recipe, the LLM itself is handed a set of tools and decides, step by step, what to do next based on what it discovers along the way."

Everyday example: a chain built for 'summarize this document' does the same three steps for every document, every time. An agent asked 'are we profitable this quarter?' might decide to query a database, realize it needs last quarter's numbers too, query again, then decide it needs a currency conversion, and only then answer — a different number of steps depending on what it finds, the way an assistant sent to the kitchen with 'make dinner' figures out the steps themselves instead of following a printed recipe."

🎯 **Standard Interview Answer:** "A chain is a deterministic, pre-wired sequence of steps executed in the same order every time, typically expressed in LCEL as `prompt | model | parser`. An agent is non-deterministic: the LLM decides, in a loop, which tool to call next based on intermediate results, continuing until it determines a stop condition is met — the **ReAct** pattern (Reason, then Act) is the classic implementation of that loop."

 **Interview Q2:** "Why did LangChain move away from classes like `LLMChain` toward LCEL's pipe syntax — what was actually wrong with the old way?"

✅ **Strong answer:** "LCEL lets you describe a pipeline's shape once instead of writing every step by hand — but the *interview* framing is really 'why did this matter enough to change the whole framework?' The old `LLMChain` - style classes were rigid bundles: if you wanted streaming, or to run many inputs at once, or to swap one piece for another, you often had to fight the class's fixed shape. LCEL pieces are all built from the same underlying interface, so *any* two pieces can be piped together, and streaming/batching/async all come for free on *any* chain you build, not just ones someone specifically coded that support for."

Everyday example: it's like the difference between buying one fixed combo meal (the old class) versus building your own meal from interchangeable parts at a buffet (LCEL) — every part at the buffet works with every other part, because they all follow the same tray-and-plate interface."

🎯 **Standard Interview Answer:** "Legacy chain classes like `LLMChain` each hard-coded their own execution logic, so features like streaming, batching, and async had to be re-implemented per class and components couldn't be freely recombined. LCEL is built on a single shared abstraction, the **Runnable interface** — every prompt, model, retriever, and parser implements `.invoke()`, `.stream()`, `.batch()`, and `.ainvoke()` — so any Runnable can be composed with any other via `|`, and those production capabilities are inherited automatically rather than hand-built per chain."

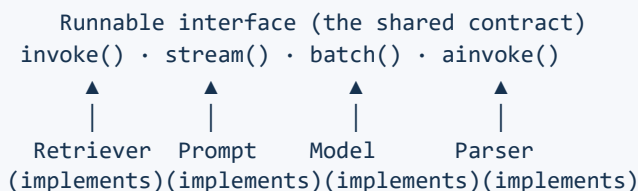
A concrete example of the actual pain, before LCEL existed: say you had a working `LLMChain` (prompt + model) and now wanted to add document retrieval in front of it. You couldn't just "add a step" to your existing chain object — retrieval-augmented Q&A needed a completely separate, purpose-built class, `RetrievalQA`, written and maintained on its own:

```
# OLD WORLD – every useful combination of steps needed its OWN
# purpose-built class, hard-coding exactly that one combination:
qa_chain = LLMChain(llm=model, prompt=prompt) # combo #1: prompt + model
retrieval_qa = RetrievalQA.from_chain_type(llm=model, retriever=my_retriever) # combo #2: retriever + prompt + model
# Want a THIRD combination the library didn't anticipate? You need
# a new class, or hand-written glue code stitching the two together.

# LCEL WORLD – there is only ever one mechanism: pipe pieces together.
# Adding retrieval is just... inserting another step:
chain = my_retriever | prompt | model | parser
```

Visually, this is the difference between one shared contract and a pile of one-off classes:

LCEL – one shared interface, everything plugs into everything



Any two pieces above can be piped with ``|`` – LangChain never needs to know what's on either side, only that both sides honor the same contract.

Legacy chains – a separate hard-coded class per combination

<code>LLMChain</code>	<code>= prompt + model</code>	(combo #1, fixed)
<code>RetrievalQA</code>	<code>= retriever + prompt + model</code>	(combo #2, fixed)
<code>ConversationChain</code>	<code>= memory + prompt + model</code>	(combo #3, fixed)

Each class bakes in its own specific steps. None of them can be recombined with each other without a new class being written.

Java analogy, since LCEL's abstraction is literally named `Runnable`: this is the exact same idea as Java's own `Runnable` interface. `Thread` and `ExecutorService` don't care what concrete class you hand them — only that it implements `Runnable` and has a `run()` method. Because of that one shared contract, *any* class implementing `Runnable` works everywhere a `Runnable` is expected, with zero special-case code needed per class.

Legacy LangChain chains were like a Java codebase that, instead of one `Runnable` interface, had a `ThreadForLoggingTask`, a `ThreadForNetworkTask`, and a `ThreadForDatabaseTask` — each its own concrete class hard-coding one specific job, unable to be mixed and matched, and needing a brand-new class every time a new combination was needed. LCEL is LangChain finally applying the same principle Java has always encouraged — **program to an interface, not an implementation** — so a prompt, a model, a retriever, and a parser are just four classes implementing one shared interface, freely swappable and composable, the same way any two `Runnable` -implementing classes can drop into any Java API expecting a `Runnable`.

 **Interview Q3:** "What are the different memory types in LangChain, and what's the trade-off between them?"

✅ **Strong answer:** "Memory is how a chatbot remembers earlier parts of a conversation, and the different types trade off *how much* it remembers against *how expensive* remembering is."

Everyday example: `ConversationBufferMemory` is like keeping a full, word-for-word transcript of everything said so far and handing the whole transcript to the model every single time — accurate, but it gets long and expensive fast, the longer the conversation runs. `ConversationSummaryMemory` is like keeping a running summary instead — 'so far, the customer asked about a refund and mentioned they're a premium member' — much shorter and cheaper to pass along every time, but small details can get smoothed away in the summarizing.

```
ConversationBufferMemory: [msg1][msg2][msg3][msg4][msg5] → all sent, every time (grows and grows)
ConversationSummaryMemory: [running summary] → short and stable, but loses fine detail
```

In practice, most teams start with the simple full-transcript approach and only switch to summarizing once the conversation gets long enough that cost or the context window actually becomes a problem."

🔗 **Standard Interview Answer:** "`ConversationBufferMemory` stores the full conversation verbatim and replays it on every call — simple and accurate, but token usage and cost grow linearly with conversation length and can eventually exceed the context window. `ConversationSummaryMemory` periodically condenses earlier turns into a running summary via an LLM call, keeping token usage roughly flat at the cost of losing fine-grained detail. `ConversationBufferWindowMemory` is a middle ground, keeping only the last k turns verbatim. The standard production guidance is to start with the buffer approach and only move to summarization once cost or context-window pressure actually shows up."

🗣️ **Interview Q4:** "Walk me through what actually happens when an LLM 'calls a tool' in LangChain."

✅ **Strong answer:** "The model itself never directly runs any code — it just *asks* for a tool to be run, and your program is the one that actually runs it."

Everyday example: think of a customer calling a support line and saying 'can you check my order status?' The support agent (the LLM) doesn't personally go into the warehouse system — they write a request slip ('look up order #4521') and hand it to a colleague (your code) who actually queries the system and reports back the result. The agent then uses that result to keep talking to the customer.

1. LLM decides it needs a tool → returns a request: `"call get_order_status(id=4521)"`
2. YOUR CODE actually runs that function, gets the real result
3. The result gets added back into the conversation as a "tool response"
4. LLM reads that result and continues — answers the user, or asks for another tool

This loop repeats until the model decides it has everything it needs to give a final answer."

🔗 **Standard Interview Answer:** "When a tool-calling-capable model receives a request alongside tool definitions, it may return an `AIMessage` containing `tool_calls` instead of a plain text response. The application code — not the model — executes the corresponding function and appends the result back into the message list as a `ToolMessage`. The model is invoked again with this updated history, and the cycle repeats until it returns a response with no further `tool_calls`."

🗣️ **Interview Q5:** "I've heard `AgentExecutor` is deprecated. What replaced it, and why should I care?"

✅ **Strong answer:** "For a while, LangChain had a few different, slightly inconsistent ways to build an agent — `initialize_agent`, `AgentExecutor`, and `create_react_agent` from `LangGraph` all did roughly the same job in different ways. As of 2026, all three have been folded into one single, unified entry point: `create_agent`."

Everyday example: it's like a company that used to have three separate forms for requesting time off — one for each department — finally replacing all of them with a single standard form that works everywhere. Same underlying process, one consistent way to do it, and the old forms still technically work for now but are being phased out."

 **Standard Interview Answer:** " `AgentExecutor` (and `initialize_agent`, and LangGraph's `create_react_agent`) are all in maintenance mode as of 2026, moved into `langchain-classic` for backward compatibility. The unified replacement is `create_agent` from `langchain.agents`, which — under the hood — runs on the LangGraph runtime rather than the old `AgentExecutor` loop. For new projects, the guidance is `create_agent` for standard cases, or building directly on LangGraph when full control over state and execution flow is needed."

 **Interview Q6:** "How would you get an LLM to return a reliably structured response, like JSON matching a specific schema, instead of free-form text?"

✓ **Strong answer:** "You describe the shape you want ahead of time, and let LangChain enforce it, rather than hoping the model's plain-text answer happens to be parseable."

Everyday example: it's the difference between asking someone to 'tell me about the product' in their own words versus handing them a form with labeled boxes — name, price, in stock (yes/no) — and asking them to fill it in. The form (a schema, usually a **Pydantic model** in Python) makes the answer predictable and easy for your code to read automatically, instead of having to guess how to parse a paragraph."

 **Standard Interview Answer:** "Define the desired shape as a Pydantic model and use structured-output support — either a model's native tool/function-calling to force schema-conformant output, or an output parser (like `PydanticOutputParser`) chained after the model via LCEL, which also injects formatting instructions into the prompt. This avoids brittle regex or string-matching against free-form text and fails fast with a validation error if the model's output doesn't conform."

 **Interview Q7:** "What's a production failure mode with LangChain that catches people off guard?"

✓ **Strong answer:** "The most common one is letting conversation memory grow forever. If you're using a simple full-transcript memory and never trim it, a long-running conversation eventually produces a prompt too big for the model's context window — and depending on the app, that shows up either as an outright error, or as the model silently 'forgetting' the earliest parts of the conversation because they got cut off."

Everyday example: it's like trying to hand someone your entire year's email inbox every time you ask them a simple question — eventually it just doesn't fit, and either the request fails outright, or they only get to read the most recent emails and never see the older, possibly important ones."

 **Standard Interview Answer:** "Unbounded conversation memory is a classic production failure: token usage grows linearly with conversation length until it exceeds the model's context window, causing either a hard error or silent truncation of earlier context. The fix is proactive — a windowed or summarizing memory strategy, explicit token-count monitoring, and a defined policy for what happens as a conversation approaches the context limit, rather than discovering the failure in production."

 **Interview Q8:** "What is LangGraph, and what problem does it solve that LangChain's chains couldn't?"

✅ **Strong answer:** "A chain always moves forward in a straight line — step 1, then step 2, then step 3, done. LangGraph exists for workflows that need to **loop back**, **branch**, or **pause and wait** — none of which a straight-line chain can naturally do.

Everyday example: think of a chain as following a recipe start to finish. LangGraph is more like a board game with branching paths — you might land on a square that sends you back three spaces to try again, or one that says 'wait here until another player makes a decision.' Agentic workflows genuinely need that: 'try an answer, check it, if it's wrong try again, if you're stuck ask a human' is a loop, not a straight line, and a chain has no way to express 'go back and retry.'"

🔗 **Standard Interview Answer:** "LangChain's chains (and LCEL generally) are fundamentally DAG-based — directed acyclic graphs, meaning execution can only move forward, never cycle back. LangGraph models workflows as a **stateful graph** that explicitly supports cycles, so patterns like retry loops, self-correction, and multi-turn agent reasoning — which require revisiting earlier steps — become expressible in a way a pure chain cannot represent."

🎙️ **Interview Q9:** "Explain LangGraph's core building blocks — state, nodes, and edges — in simple terms."

✅ **Strong answer:** "Think of a relay race with a baton that gets written on at every station.

```
STATE (the baton) → a shared notebook every node can read and update
NODE (a runner)   → a function (often an LLM call) that does one job,
                    then updates the notebook before passing it on
EDGE (the handoff) → the rule for which runner gets the baton next
```

Everyday example: each runner (node) reads what's written in the notebook (state) so far, does their part of the race, jots down what they learned or decided, then hands the baton to the next runner as directed by the course markings (edges). Unlike a normal relay, though, the course can send the baton *back* to an earlier runner if a marking says so — that's the cycle a plain chain can't do."

🔗 **Standard Interview Answer:** "**State** is a typed schema (often a `TypedDict` or Pydantic model) that flows through the entire graph and gets read and updated by every node. A **node** is a unit of computation — usually a Python function or an LLM call — that receives the current state and returns updates to it. An **edge** defines the control flow between nodes; edges can be fixed or **conditional**, where a function inspects the current state and decides which node to route to next, which is what enables branching and loops."

💡 **Interview Q9 (follow-up):** "If a single node has multiple outgoing edges, how does it actually decide which one to follow? And what determines when the graph stops looping entirely?"

✅ **Strong answer:** "The node itself doesn't 'choose' between edges by some built-in magic. Continuing the relay race: right after a runner finishes their leg, a referee standing at that exact spot (a routing function) looks at what's currently written in the notebook (the state) and points to exactly *one* of the possible next runners — never more than one at a time. The node doesn't decide; a small decision-function decides, and it always hands back exactly one answer.

For stopping: the race track has a special finish line, called **END** in LangGraph. Any referee, at any point, can point to the finish line instead of another runner, and once the baton reaches it, the race is over — that's the termination condition, and it's just another destination an edge can point to, nothing more exotic than that. The real danger is a referee who *never* points to the finish line — the race would go forever — which is why real setups add a safety rule too: 'after 3 laps with no clear winner, call it anyway.'"

 **Standard Interview Answer:** "Multiple potential next-steps from one node are declared via a **conditional edge**, attached with `add_conditional_edges`, which maps the possible outputs of a routing function to specific node names. The routing function — a plain Python function, or an LLM call — inspects the current state and returns exactly one value: either a node name, or the reserved `END` sentinel from `langgraph.graph`. That single returned value determines which one path is taken for that execution; the other edges remain possible but untraveled that round. The graph terminates the moment execution reaches `END`, which any fixed or conditional edge can point to. Since a routing function could in principle loop forever if its logic never resolves to `END`, production graphs typically pair conditional loops with a hard iteration cap tracked in the state itself (e.g. a `retry_count` field the routing function checks), guaranteeing termination even if the 'ideal' stopping condition is never cleanly met."

 **Interview Q10:** "How does routing actually work in LangGraph — how does the graph decide which node runs next?"

 **Strong answer:** "Two ways: a fixed path, or a decision made on the fly."

Everyday example: a fixed edge is like a hallway with only one door at the end — you always go through it. A conditional edge is like a hallway with a person standing at a fork, checking something about you (what's currently written in the shared notebook) and pointing you left or right based on that — maybe 'if the answer looks confident, go straight to the exit; if not, go back and try again.'"

 **Standard Interview Answer:** "A regular edge unconditionally connects one node's output to the next node. A **conditional edge** attaches a routing function to a node's output — that function inspects the current state and returns the name of whichever node should execute next, enabling branching, retries, and cycles based on runtime conditions rather than a fixed sequence."

 **Interview Q11:** "What is a checkpoint in LangGraph, and why does it matter?"

 **Strong answer:** "It's what lets a workflow pause, get interrupted, or crash — and pick back up later exactly where it left off, instead of starting over."

Everyday example: it's the autosave feature in a video game. Without it, if your console loses power mid-level, you're back at the very beginning. With autosave, you resume from your last checkpoint, with all your progress intact. A LangGraph checkpointer does the same thing for an agent's state — saving a snapshot after each step so the whole run can be resumed later, even by a different process."

 **Standard Interview Answer:** "A checkpointer is a persistence layer, backed by something like SQLite, Postgres, or Redis, attached to the graph at compile time. It saves a snapshot of the graph's state after every step (a 'superstep'), keyed by a thread ID. This enables resuming execution after a crash, running multiple conversations concurrently without state bleeding between them, and is the mechanism that underlies human-in-the-loop interrupts."

 **Interview Q11 (follow-up):** "Is the checkpointer defined per node, or once for the whole graph? Concretely: in a 5-node graph where node 4 needs human-in-the-loop approval, once node 3 finishes, is there one snapshot of the whole 5-node graph, currently pointing at node 4 waiting for approval?"

 **Strong answer:** "Once for the whole graph — you attach it a single time at compile time (`graph.compile(checkpointer=...)`), and it then applies automatically to every node with no per-node setup at all."

On the scenario: close, but it's not one snapshot 'of the whole graph.' Back to the video-game autosave — the game quietly writes a *new* save file after every level you clear, it doesn't keep one save file describing the entire game. By the time node 3 finishes, three separate snapshots already exist, one written after each of nodes 1, 2, and 3. The *latest* of those is the one that matters: it carries all the state accumulated through node 3, tagged with 'next up: node 4,' and that's exactly the save file execution resumes from once a human approves."

 **Standard Interview Answer:** "The checkpointer is registered once, at graph compilation, not per node — it then transparently persists a state snapshot after every superstep for the lifetime of that thread. In a 5-node graph with node 4 as an interrupt point, three checkpoints already exist by the time node 3 completes, one per completed superstep, all tied to the same thread ID. The most recent checkpoint reflects the state after node 3 plus the graph's current position (node 4, paused) — resuming loads exactly that checkpoint rather than restarting the run."

 **Interview Q12:** "How would you build a human-in-the-loop approval step into an agent workflow?"

 **Strong answer:** "You pause the graph right before the risky step, show a human what the agent is about to do, and only continue once they approve — using the same checkpointing mechanism that lets a graph resume after any pause."

Everyday example: think of a junior employee drafting an email and a manager who has to approve it before it actually gets sent. The draft (the graph's state) sits waiting; nothing moves forward until the manager says go — and once they do, the process picks up exactly where it paused, rather than starting the whole draft over."

 **Standard Interview Answer:** "LangGraph supports interrupting execution at a specific node — commonly right before an irreversible action — using its checkpointing system to persist state at that pause point. Execution resumes only when the application explicitly continues the run, typically after collecting human approval or edited input, at which point the graph proceeds from the saved state rather than re-running from the start."

 **Interview Q13:** "If I need multiple specialized agents working together, how would LangGraph structure that?"

 **Strong answer:** "The most common pattern is having one 'manager' agent that doesn't do the actual work itself, but instead looks at the task and decides which specialist agent should handle it next."

Everyday example: think of a team lead who doesn't write code, run tests, or write documentation themselves — their whole job is reading each request and routing it to the right specialist: 'this one's a bug, send it to the engineer; this one needs docs, send it to the writer.' Each specialist reports back, and the lead decides what happens next — hand it to another specialist, or wrap up and respond."

 **Standard Interview Answer:** "The **supervisor pattern** is the standard approach: a dedicated routing node uses structured output — the LLM returns a schema-conformant decision, not a string to parse — to decide which specialist agent (each modeled as its own node or sub-graph) should act next. An alternative is the **swarm** pattern, where agents hand control directly to each other without a central router. Both commonly share state across agents by default, so every agent reads from and writes to the same state channels unless explicitly isolated."

 **Interview Q13 (follow-up):** "So a node is one computational unit, and that unit can host anything — an LLM call, or arbitrary Python code. Does that mean one node is equivalent to one agent?"

✅ **Strong answer:** "You've got the node part exactly right — a node is just a Python function (or a compiled graph), and it can genuinely hold *anything* inside it: an LLM call, a database query, a plain calculation, or nothing but print statements. LangGraph places zero restrictions on what a node does internally.

On 'node = agent': correct in the simple case, but with a nuance worth knowing. Sometimes a specialist agent really is just one node — one LLM call configured with a persona and instructions for that specialty. But a genuinely capable agent, one that reasons, calls a tool, reads the result, and decides whether to try again, is usually itself a small **sub-graph** with several internal nodes of its own — and LangGraph lets you take that whole compiled sub-graph and drop it into the supervisor's graph as if it were a single node. Either way, the supervisor never has to care which one it's looking at: from its point of view, routing always happens at the granularity of 'one node per agent,' whether that node hides one line of code or an entire internal loop.

Concrete example: a coding-assistant supervisor with three specialists — a *Research Agent*, a *Coding Agent*, and a *Testing Agent*. The supervisor is one routing node: it looks at the task and returns 'coding' or 'testing' as its decision. The *Testing Agent* might genuinely be one plain node that just runs a test suite and reports pass/fail. But the *Coding Agent* is likely a whole sub-graph on its own: reason about the fix (node) → write and execute the code (node) → read the result (node) → loop back to reasoning if it didn't work. From the supervisor's perspective, though, that entire loop is still just 'the coding node' — one thing to route to, same as the simple testing node."

🌀 **Standard Interview Answer:** "A node is an arbitrary unit of computation with no constraints on its contents. Whether a node maps one-to-one with an 'agent' depends on the agent's complexity: a lightweight specialist can be a single node wrapping one LLM call, while a fully agentic specialist — one running its own reason-act-observe loop — is typically implemented as a compiled sub-graph and then embedded as a single node within the parent (supervisor) graph, since LangGraph supports nesting a compiled graph as a node. This preserves a uniform routing interface for the supervisor: it always routes at node granularity, regardless of how much internal complexity a given node encapsulates."

Sources consulted while calibrating this section: - [LangChain Software Engineer Interview Guide \(2026\)](#) — [Exponent](#) - [Top LangChain Interview Questions and Answers for 2026](#) — [DataCamp](#) - [LangChain Interview Questions: Real Production Probes \(2026\)](#) — [Interview Baba](#) - [Is AgentExecutor Deprecated in LangChain? The Real State of LangChain Agent APIs in 2026](#) — [BSWEN](#) - [LangChain Agents in 2026: The Complete Guide \(Updated for LangGraph Era\)](#) — [EasyClaw](#) - [Conversational Memory in LangChain](#) — [Aurelio AI](#) - [What is LangGraph? Stateful Agent Graphs Explained in 2026](#) — [FutureAGI](#) - [LangGraph State: Checkpoints, Threads, and Recovery](#) — [Easton](#) - [Top 35 LangGraph Interview Questions \(2026\)](#) — [Interview Coder](#) - [Multi-Agent Orchestration in LangGraph: Supervisor vs Swarm](#) — [Focused](#)